

Отладка и логирование Node.js-приложений

Debugger, stack trace, structured logging, request ID и диагностика ошибок

Результаты занятия

- ✓ Различать debugging и logging
- ✓ Читать stack trace Node.js
- ✓ Использовать node --inspect и breakpoints
- ✓ Проверять значения переменных во время выполнения
- ✓ Выбирать уровни логирования
- ✓ Использовать structured JSON logging
- ✓ Добавлять request ID к HTTP-запросам
- ✓ Логировать ошибки без утечки секретов
- ✓ Диагностировать HTTP и Prisma failures
- ✓ Организовывать воспроизводимый troubleshooting workflow

Учебные вопросы

- | | | |
|---|---------------------|--|
| 1 | Как искать ошибку | Симптом, воспроизведение, stack trace и гипотеза |
| 2 | Debugger Node.js | --inspect, breakpoint, step и variables |
| 3 | Логирование | Уровни, structured logs и Pino |
| 4 | Request tracing | Request ID и child logger |
| 5 | Диагностика backend | HTTP, Prisma, environment и production-safe logs |



Учебный вопрос 1. Как искать ошибку

Отладка начинается не с исправления кода, а с воспроизведения проблемы

Debugging и logging

- Debugging — интерактивный поиск причины ошибки во время выполнения.
- Logging — запись диагностических событий приложения.
- Debugger удобен при локальной разработке.
- Logs особенно важны, когда ошибка произошла без разработчика рядом.
- Оба инструмента дополняют tests, но не заменяют их.

Debugger и Logs

Debugger

- Останавливает выполнение
- Показывает variables и call stack
- Позволяет выполнять код step-by-step
- Лучше для локального поиска причины

Logs

- Не останавливают приложение
- Сохраняют события во времени
- Работают в production
- Помогают анализировать уже произошедшую ошибку

Базовый troubleshooting workflow

- 1 Воспроизвести** Получить тот же failure на известном request/input.
- 2 Собрать факты** Status, response, stack trace, logs и environment.
- 3 Локализовать** Определить слой: middleware, Controller, Service, Prisma или DB.
- 4 Проверить гипотезу** Использовать debugger, дополнительные logs или test.
- 5 Исправить и проверить** Повторить scenario и запустить regression tests.

Troubleshooting: symptom → reproduce → evidence → hypothesis → fix → regression test



От проблемы — к стабильному и надежному решению

Stack trace

- Stack trace показывает цепочку вызовов функций от места возникновения ошибки до верхнего уровня выполнения.
- Первая полезная строка часто указывает место throw/failure.
- Далее видны вызывающие функции.
- Строки из `node_modules` обычно являются контекстом, а не обязательно причиной.
- Сначала ищите собственные файлы проекта и конкретную строку.

Пример stack trace

TEXT

```
Error: Product not found  
  at getProductById (services/productService.js:18:11)  
  at async getProduct (controllers/productController.js:22:21)  
  at async Layer.handleRequest (.../express/...)
```

Не путайте симптом и причину

- 500 Internal Server Error — симптом на HTTP-уровне.
- Причина может быть в invalid data, Prisma query, environment variable или bug Service.
- Сообщение клиента должно быть безопасным и коротким.
- Подробности нужны в server logs и debugger.



Учебный вопрос 2. Debugger Node.js

Inspector позволяет остановить программу и исследовать её состояние

Node.js Inspector

- Node.js поддерживает V8 Inspector.
- `--inspect` запускает inspector и сразу выполняет приложение.
- `--inspect-brk` останавливается на первой строке.
- По умолчанию inspector использует loopback interface и стандартный debug port.
- К inspector можно подключить IDE или Chrome DevTools.

Запуск Inspector

BASH

```
node --inspect server.js
```

```
node --inspect-brk server.js
```

Node Inspector: как это работает

Запускаем **приложение** с `--inspect`, подключаемся из IDE или Chrome DevTools и отлаживаем код в работающем процессе Node.js.



Node.js Inspector использует Chrome DevTools Protocol (CDP) и **WebSocket**. Вы можете подключиться из VS Code, WebStorm или прямо в браузере (`chrome://inspect`).

1 Node.js process (ваше приложение)



Запуск с инспектором:

```
node --inspect app.js  
# или остановка на первой строке:  
node --inspect-brk app.js
```



Приложение работает, открывает debug port (по умолчанию 9229) и ждёт подключения от клиента.



```
PS C:\project> node --inspect app.js  
Debugger listening on ws://127.0.0.1:9229/  
89c1d9m9-4f36-4c2a-9c07-5e8b7e3f1b2a  
For help, see: https://nodejs.org/en/docs/  
inspector  
Server running on http://localhost:3000
```

Порт
9229
(по умолчанию)



2 Inspector protocol (Chrome DevTools Protocol)

WebSocket

`ws://127.0.0.1:9229`

Команды отладчика
(`setBreakpoint`, `step`, ...)

События из процесса
(`paused`, `debug event`, ...)

Данные (stack, variables,
console, ...)



Используется стандартный Chrome DevTools Protocol (CDP)

3 IDE / DevTools (клиент отладчика)



VS Code



WebStorm



Chrome DevTools

VS Code: `launch.json`

```
{  
  "version": "0.2.0",  
  "configurations": [  
    {  
      "type": "node",  
      "request": "attach",  
      "name": "Attach to Node.js",  
      "port": 9229  
    }  
  ]  
}
```

Chrome DevTools

- 1 Откройте в браузере `chrome://inspect`
- 2 Нажмите «Configure...» (при необходимости)
- 3 Выберите ваш процесс и нажмите «inspect»

Remote Target #LOCALHOST

Target (v22.18.0)

app.js file:///C:/project/app.js
`inspect` `pause` `focus tab`



Реальная отладка

Breakpoints, step over/into, просмотр переменных, call stack, watch.



Работает с вашим кодом

Отладка в запущенном процессе без изменений в коде приложения.



Стандартный протокол

Chrome DevTools Protocol (CDP) по WebSocket (порт 9229).



Гибкие клиенты

VS Code, WebStorm или Chrome DevTools.



Безопасность

Не используйте `--inspect` на публичных серверах. Ограничивайте доступ (localhost, VPN, firewall).



Итог: Node Inspector даёт полный контроль над выполнением Node.js-приложения через знакомые инструменты разработчика.

Breakpoint

- Breakpoint — точка, в которой debugger временно останавливает выполнение программы.
- Поставьте breakpoint перед подозрительной строкой.
- Отправьте HTTP request, который воспроизводит проблему.
- Исследуйте `req.params`, `req.body`, `req.user` и `local variables`.
- Продолжайте выполнение `step-by-step`.

Основные действия debugger

Действие	Назначение
Continue	Продолжить до следующего breakpoint
Step over	Выполнить текущую строку без входа в функцию
Step into	Войти внутрь вызываемой функции
Step out	Выйти из текущей функции
Variables	Посмотреть local/global значения
Call stack	Посмотреть цепочку вызовов
Watch	Следить за выражением

Точка для исследования Product request

JAVASCRIPT

```
export async function getProduct(req, res, next) {  
  try {  
    const id = Number(req.params.id);  
  
    debugger;  
  
    const product = await productService.getById(id);  
    res.json(product);  
  } catch (error) {  
    next(error);  
  }  
}
```

Что важно

- debugger statement создаёт программную breakpoint.
- Можно проверить id перед вызовом Service.
- После диагностики такие statements обычно удаляют.

Watch expressions

- Можно наблюдать выражение без добавления `console.log()`.
- Примеры: `req.params.id`, `Number(req.params.id)`, `req.user?.role`.
- Watch помогает сравнить ожидаемое и фактическое значение.
- Не изменяйте state случайно во время debugging.

Breakpoint workflow: от запроса до причины ошибки

Пошаговый процесс отладки Node.js-приложения с использованием breakpoint в IDE (пример: VS Code).



Breakpoint позволяет остановить выполнение кода и исследовать состояние приложения в реальном времени.

1 Request

Отправляем HTTP-запрос

Клиент (браузер, Postman или тест) отправляет запрос в наше Express-приложение.



Клиент
(браузер / Postman)

GET /api/products/1

Headers:

Authorization: Bearer ...
Content-Type: application/json



Express app
на Node.js



2 Pause

Выполнение останавливается

Когда запрос доходит до строки с breakpoint, выполнение кода приостанавливается.

```
controllers/productController.js
10 export async function
11 getProductById(req, res) {
12   const id = req.params.id;
13   const product = await
14   productService.findById(id);
15
16   if (!product) {
17     return res.status(404)
18     .json({ message: 'Not found' });
19 }
```

Код остановлен
на breakpoint
(строка 13)

3 Variables

Изучаем переменные

Проверяем значения переменных в текущей области видимости (Scope).

```
VARIABLES
Local
  req: IncomingMessage { .. }
    params: { id: '1' }
    query: {}
    body: {}
  > headers: { ... }
  > res: ServerResponse { ... }
    id: '1'
    product: undefined
Closure
  > productService: ProductService
    this: undefined
```

Можно раскрывать объекты и смотреть их содержимое в реальном времени.

4 Call stack

Смотрим стек вызовов

В Call Stack видно, как мы попали в текущую строку: цепочка вызовов функций.

```
CALL STACK
getProductById
productController.js:13
Layer.handle [as handle_request]
layer.js:95
next
route.js:149
Route.dispatch
route.js:119
handle
router/index.js:284
processParams
router/index.js:346
...
```

Стек вызовов помогает понять путь запроса через middleware и роутеры.

5 Step

Пошаговое выполнение

Используем кнопки отладки для пошагового выполнения кода и проверки логики.

```
DEBUG CONTROLS (VS Code)
▶ Продолжить (F5)
Continue
⏮ Шаг с обходом (F10)
Step Over
➡ Шаг внутрь (F11)
Step Into
⏮ Шаг наружу (Shift+F11)
Step Out
🔄 Перезапустить (Ctrl+Shift+F5)
Restart
■ Остановить (Shift+F5)
Stop
```

Шаг за шагом проверяем значения, условия и поведение кода.

6 Root cause

Находим причину

Анализируя переменные, стек вызовов и выполнение кода, находим реальную причину ошибки.



Пример причины:

```
const product = await
productService.findById(id);

// product = undefined
// потому что в БД нет
// записи с id = '1'
```

Причина найдена!
Исправляем код
и добавляем тест.



Результат

Ошибка обнаружена и исправлена.
Добавляем regression test, чтобы она не вернулась.



Советы

- Используйте осмысленные имена переменных.
- Проверяйте данные на каждом слое (controller → service → DB).
- Комбинируйте отладку с логированием.
- После исправления добавьте тест.



Хорошая отладка — это не гадание,
а исследование с помощью инструментов.





Учебный вопрос 3. Логирование

Хороший log отвечает: что произошло, где, когда и в каком контексте

Почему console.log недостаточно

- Свободный текст сложно фильтровать и анализировать автоматически.
- Нет единой структуры полей.
- Легко случайно вывести password/token.
- Сложнее связывать события одного HTTP request.
- Для production удобнее structured logging.

Уровни логирования

- trace — очень подробная диагностика.
- debug — информация для разработки.
- info — нормальные важные события.
- warn — подозрительная ситуация, приложение продолжает работу.
- error — ошибка операции/request.
- fatal — критическая ошибка перед завершением процесса.

Log levels: уровни логирования в Node.js

Иерархия уровней от подробной диагностики до критических ошибок.

Больше информации
(для разработки и диагностики)

Более серьёзные события
(требуют внимания)

1 TRACE (наиболее подробный)



Самая подробная диагностическая информация.

Когда использовать:

- Трассировка выполнения кода
- Детали алгоритмов
- Низкоуровневые события
- Диагностика сложных проблем

```
logger.trace('Start processing request', { id: 1 });
```

Пример лога:

```
{
  "level": "trace",
  "msg": "Start processing request",
  "id": 1
}
```

2 DEBUG (подробная отладка)



Информация для разработчиков при отладке.

Когда использовать:

- Значения переменных
- Детали работы функций
- Запросы к внешним сервисам
- Пошаговая диагностика приложения

```
logger.debug('SQL query', {
  query: 'SELECT * FROM products',
  params: [1] });
```

Пример лога:

```
{
  "level": "debug",
  "msg": "SQL query",
  "query": "SELECT * FROM products",
  "params": [1]
}
```

3 INFO (обычная информация)



Важные события в работе приложения.

Когда использовать:

- Успешные операции
- Запуск и остановка сервиса
- Обработка запросов
- Бизнес-события (создание, обновление, удаление данных)

```
logger.info('Product created', {
  id: 1, name: 'Laptop' });
```

Пример лога:

```
{
  "level": "info",
  "msg": "Product created",
  "id": 1,
  "name": "Laptop"
}
```

4 WARN (предупреждение)



Потенциальные проблемы, которые не приводят к сбою, но требуют внимания.

Когда использовать:

- Некорректные входные данные
- Устаревшие API
- Превышение лимитов
- Повторные попытки
- Необычное поведение системы

```
logger.warn('Rate limit exceeded', { ip: '192.168.1.1' });
```

Пример лога:

```
{
  "level": "warn",
  "msg": "Rate limit exceeded",
  "ip": "192.168.1.1"
}
```

5 ERROR (ошибка)



Ошибки, при которых операция не выполнена, но приложение продолжает работать.

Когда использовать:

- Ошибка в бизнес-логике
- Неудачные запросы к внешним сервисам
- Ошибки валидации
- Исключения, которые обрабатываются

```
logger.error('Failed to save product', { error: err.message });
```

Пример лога:

```
{
  "level": "error",
  "msg": "Failed to save product",
  "error": "Unique constraint violation"
}
```

6 FATAL (критическая ошибка)



Критические ошибки, после которых приложение не может продолжать работу.

Когда использовать:

- Не удалось подключиться к базе данных
- Критическая ошибка конфигурации
- Искчерпаны ресурсы
- Необработанное исключение
- Завершение работы сервиса

```
logger.fatal('Database connection failed', { error: err });
```

Пример лога:

```
{
  "level": "fatal",
  "msg": "Database connection failed",
  "error": "ECONNREFUSED"
}
```



Настройка уровня логов

В production обычно используют уровень INFO и выше, а более подробные уровни (TRACE, DEBUG) включают только при необходимости.



Структурированное логирование

Используйте JSON-формат (например, Pino), чтобы легко фильтровать и анализировать логи.



Практическое правило

Логите достаточно информации, чтобы решить проблему, но не перегружайте логи излишними деталями.

Structured logging

- Structured log хранит диагностическое событие как набор полей, обычно в JSON/NDJSON формате.
- Поля можно фильтровать и индексировать.
- Один формат работает локально и в production.
- Удобно добавлять requestId, userId, route, statusCode и duration.
- Сообщение остаётся человекочитаемым, а контекст — машинно обрабатываемым.

Установка Pino

BASH

```
npm install pino  
npm install --save-dev pino-pretty
```

Базовый logger

JAVASCRIPT

```
import pino from "pino";

export const logger = pino({
  level: process.env.LOG_LEVEL ?? "info",
  redact: [
    "password",
    "passwordHash",
    "accessToken",
    "req.headers.authorization"
  ]
});
```

Что важно

- level управляет минимальным уровнем log.
- redact скрывает чувствительные поля.
- В production удобно оставлять JSON output.

Structured log event

JAVASCRIPT

```
logger.info({  
  productId: product.id,  
  categoryId: product.categoryId  
}, "Product loaded");
```

Плохой и хороший log

Плохо

- `console.log("loaded " + id)`
- Непонятная структура
- Сложно фильтровать
- Легко смешать secrets и user input

Лучше

- `logger.info({ productId: id }, "Product loaded")`
- Стабильные поля
- Уровень события
- Удобно искать по productId



Учебный вопрос 4. Request tracing

Request ID связывает события одного HTTP-запроса между слоями

Зачем request ID

- Одновременно сервер обрабатывает много requests.
- Logs разных requests перемешиваются.
- Уникальный requestId позволяет собрать одну цепочку событий.
- Один ID можно передавать из middleware в Controller и Service.
- При необходимости requestId возвращают клиенту в response header.

Request ID middleware

JAVASCRIPT

```
import { randomUUID } from "node:crypto";
import { logger } from "../config/logger.js";

export function requestContext(req, res, next) {
  const requestId = req.get("X-Request-Id") ?? randomUUID();

  req.requestId = requestId;
  req.log = logger.child({ requestId });
  res.set("X-Request-Id", requestId);

  next();
}
```

Что важно

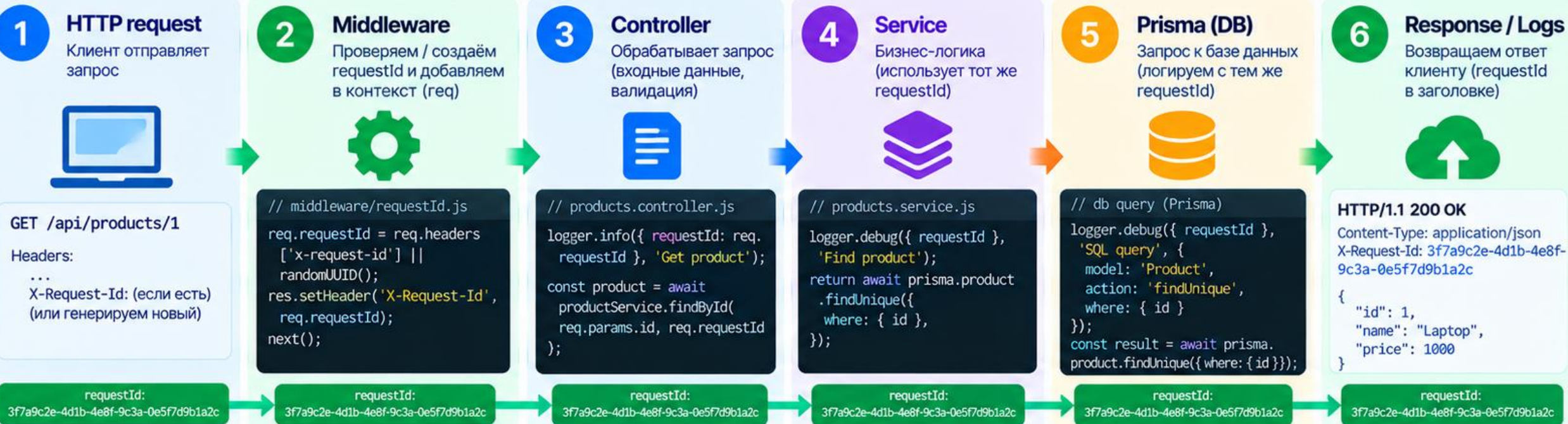
- randomUUID() создаёт ID, если клиент его не передал.
- child logger автоматически добавляет requestId к каждой записи.
- Один request можно найти во всех слоях.

Request ID: один requestId на весь путь запроса

Один и тот же requestId сопровождает запрос через все слои приложения — от HTTP до базы данных — и позволяет быстро находить связанные логи.



Request ID связывает все события одного запроса в единый трейс. Упрощает отладку, мониторинг и анализ проблем.



Примеры логов (все записи содержат одинаковый requestId)

```
2026-09-21 10:15:23.001
{
  "level": "info",
  "requestId": "3f7a9c2e...",
  "msg": "Incoming request",
  "method": "GET",
  "url": "/api/products/1"
}
```

```
2026-09-21 10:15:23.002
{
  "level": "debug",
  "requestId": "3f7a9c2e...",
  "msg": "Request ID set",
  "requestId": "3f7a9c2e..."
}
```

```
2026-09-21 10:15:23.010
{
  "level": "info",
  "requestId": "3f7a9c2e...",
  "msg": "Get product",
  "productId": "1"
}
```

```
2026-09-21 10:15:23.015
{
  "level": "debug",
  "requestId": "3f7a9c2e...",
  "msg": "Find product",
  "productId": "1"
}
```

```
2026-09-21 10:15:23.020
{
  "level": "debug",
  "requestId": "3f7a9c2e...",
  "msg": "SQL query",
  "model": "Product",
  "action": "findUnique"
}
```

```
2026-09-21 10:15:23.045
{
  "level": "info",
  "requestId": "3f7a9c2e...",
  "msg": "Request completed",
  "status": 200,
  "durationMs": 44
}
```



Быстрая диагностика
Легко найти все логи одного запроса.



Трассировка
Видно полный путь запроса через все слои приложения.



Мониторинг
Удобно анализировать ошибки и производительность.



Поддержка и эксплуатация
Проще разбирать инциденты в продакшене.



Один requestId —
вся история запроса.



Использование child logger

JAVASCRIPT

```
req.log.info({  
  method: req.method,  
  path: req.originalUrl  
}, "Request started");  
  
req.log.info({  
  statusCode: res.statusCode  
}, "Request completed");
```

Что полезно логировать для HTTP request

- requestId.
- method и route/path.
- statusCode.
- duration.
- userId — если это действительно нужно и допустимо.
- Ключевые domain IDs: productId, orderId.
- Error type/code — без чувствительных данных.

Что нельзя логировать без необходимости

- Plaintext passwords.
- Password hashes.
- JWT access/refresh tokens.
- Authorization header целиком.
- Session cookies.
- Полные payment/card данные.
- Secrets из environment variables.
- Большие или непроверенные user objects.



Учебный вопрос 5. Диагностика backend

Ошибки HTTP, Prisma и environment должны оставлять понятные диагностические следы

Error logging middleware

JAVASCRIPT

```
export function errorHandler(error, req, res, next) {
  req.log?.error({
    err: error,
    method: req.method,
    path: req.originalUrl
  }, "Request failed");

  const status = error.status ?? 500;

  res.status(status).json({
    error: status < 500
      ? error.message
      : "Internal server error",
    requestId: req.requestId
  });
}
```

Что важно

- Server log получает Error object и stack trace.
- Клиент не получает внутренний stack.
- requestId помогает связать response и server logs.

Безопасный поток обработки ошибки

Внутренняя ошибка и stack trace остаются на сервере, а клиент получает только безопасный ответ: status, message и requestId.



Один requestId — весь путь от запроса до лога. Это упрощает диагностику и не раскрывает внутренние детали клиенту.



СЕРВЕР (Node.js / Express)

Обработка запроса, возникновение ошибки и её логирование



HTTP Request

Клиент отправляет запрос на сервер

GET /api/products/1

Headers:

X-Request-Id: req-7f3a9c
Content-Type: application/json

Запрос получает уникальный requestId (например, из заголовка или генерируется на сервере).



Middleware / Controller / Service

Запрос проходит через middleware, контроллер и сервисы

```
// controller.js
app.get('/api/products/:id',
  async (req, res, next) => {
    try {
      const product = await
        productService.findById(req.params.id);
      res.json(product);
    } catch (err) {
      next(err); // передаём
    }
  })
```

Если в процессе возникает ошибка — она передаётся в обработчик.



Внутренняя ошибка

В приложении происходит ошибка (исключение)

```
// пример объекта ошибки
Error: Database connection failed:
    at ProductService.findById
    (/app/src/services/product.js:42:17)
    at processTicksAndRejections
    (node:internal/process/task_queues:95:5)
{
  name: 'Error',
  message: 'Database connection failed',
  stack: 'Error: Database connection failed\n...',
  file: 'product.js',
  line: 42,
  requestId: 'req-7f3a9c'
}
```

Содержит технические детали: объект ошибки, stack trace, файл/строку, внутреннее сообщение и requestId.



Error Handler

Глобальный обработчик ошибок (middleware)

```
// errorHandler.js
app.use((err, req, res, next) => {
  const requestId = req.requestId;

  logger.error({
    requestId,
    error: err,
    layer: 'product-service'
  });

  const status = err.status || 500;
  const message = 'Internal Server Error';

  res.status(status).json({
    status,
    message,
    requestId
  });
});
```

Логировать полную ошибку на сервере и отправляет клиенту безопасный ответ без внутренних деталей.



Логи сервера

Полная техническая информация сохраняется в логах

```
{
  timestamp: '2026-09-21T10:15:23.045Z',
  level: 'error',
  requestId: 'req-7f3a9c',
  status: 500,
  error: 'Database connection failed',
  name: 'Error',
  stack: 'Error: Database connection...\n at ...',
  layer: 'product-service',
  file: 'product.js',
  line: 42
}
```



ОТВЕТ КЛИЕНТУ

Клиент получает только безопасную информацию

```
{
  "status": 500,
  "message": "Internal Server Error",
  "requestId": "req-7f3a9c"
}
```

Клиент получает понятное сообщение об ошибке, но без внутренних деталей приложения.



Stack trace только на сервере



Не отправляем клиенту

Эти данные могут содержать конфиденциальную информацию и не должны попадать в ответ клиенту.

- Stack trace (трассировка стека)
- Пути к файлам (например, /app/src/...)
- SQL-запросы и детали базы данных
- Внутренние сообщения об ошибке
- Целый объект ошибки (err)
- Секретные значения (пароли, токены, ключи)



Это может раскрыть внутреннюю структуру приложения и создать риски безопасности.



Преимущества такого подхода

- ✓ **Безопасность** — не раскрываем внутренние детали.
- ✓ **Удобная диагностика** — полные ошибки в логах сервера.
- ✓ **Связь ответа с логами через requestId** — легко найти нужную запись.
- ✓ **Клиент получает понятное сообщение** — можно корректно обработать ошибку.



Безопасность и удобная отладка в одном решении!



requestId связывает всё

Один и тот же requestId есть в:

- ✓ входящем запросе (заголовок или сгенерирован)
- ✓ записи в логах сервера
- ✓ ответе клиенту

По requestId можно быстро найти нужную запись в логах и понять, что именно произошло.

Диагностика Prisma errors

- Логируйте тип/код ошибки и безопасный context.
- Не отправляйте внутренний Prisma stack клиенту.
- Проверьте DATABASE_URL и доступность PostgreSQL.
- Отделяйте business 404/409 от неожиданного 500.
- Integration test должен воспроизводить важные failure scenarios.

HTTP symptom → где искать

Симптом	Возможный слой	Что проверить
400	Validation	req.body/params, rules
401	Authentication	Bearer token, JWT secret
403	Authorization	req.user.role, ownership
404	Router/Service	route params, DB record
409	Business/DB	unique constraint, state conflict
500	Service/Prisma/Environment	stack trace, logs, DB connection

Environment diagnostics

- Проверьте `NODE_ENV`.
- Проверьте наличие обязательных `environment variables`.
- Не печатайте `secret values` целиком.
- Логируйте безопасные признаки конфигурации: `environment`, `port`, `service version`.
- Ошибочная конфигурация должна обнаруживаться при `startup`, а не после первого `request`.

Startup diagnostics

JAVASCRIPT

```
logger.info({  
  nodeEnv: process.env.NODE_ENV,  
  port: process.env.PORT ?? 3000,  
  databaseConfigured: Boolean(process.env.DATABASE_URL)  
}, "Application starting");
```

Диагностика 500: от requestId к причине и исправлению

Схема реального расследования backend failure

1 requestId

Клиент видит ошибку 500 с requestId

GET /api/products/123

HTTP/1.1 500

Internal Server Error

```
{
  "error": "Internal Server Error",
  "requestId": "req-7f3a9c"
}
```

requestId связывает видимую клиенту ошибку с логами на сервере.

2 error log

Находим в логах запись по requestId

```
{
  "timestamp": "2024-05-18T10:24:32Z",
  "level": "error",
  "status": 500,
  "requestId": "req-7f3a9c",
  "message": "Failed to fetch product",
  "userId": "usr-42",
  "path": "/api/products/123"
}
```

Сначала находим соответствующую запись в логах по requestId.

3 stack trace

Смотрим стек вызовов, чтобы понять, где распространилась ошибка

```
Error: Failed to fetch product
at ProductService.getProduct (productService.js:42:17)
at ProductController.getById (productController.js:18:5)
at Layer.handle [as handle_request] (router/layer.js:95:5)
at next (router/route.js:149:13)
at asyncHandler (middleware/asyncHandler.js:12:9)
...
```

Стек вызовов показывает, где именно ошибка распространилась (файл и строка).

4 layer

Определяем, к какому слою приложения относится ошибка



Middleware

логирование, auth, валидация



Controller

обработка запросов, валидация данных



Service

бизнес-логика, работа с данными



Prisma / PostgreSQL

доступ к базе данных



По стеку определяем слой, например, ошибка в слое Service.

5 root cause

Выявляем настоящую причину дефекта



Причина:
Ошибка подключения к базе данных

```
PrismaClientInitializationError:
Can't reach database server
at "db:5432"
  at new PrismaClient
    (node_modules/@prisma/client/...)
  at productService.js:42:17
...
```



Здесь находим реальный дефект и его конкретную причину.

6 regression test

Пишем тест, который воспроизводит ошибку и предотвращает её в будущем

product.e2e.test.js

```
const request = require('supertest');
const app = require('../app');

describe('Product API', () => {
  it('возвращает 500, если БД недоступна', async () => {
    jest.spyOn(prisma, '$connect')
      .mockRejectedValueOnce(
        new Error('Connection failed')
      );
    const res = await request(app)
      .get('/api/products/123');
    expect(res.status).toBe(500);
    expect(res.body.requestId)
      .toBeDefined();
  });
});
```



PASS — тест проходит после исправления

Добавляем regression test

Фиксим ошибку (code change)



Приложение
(с ошибкой)



Стабильное приложение
Ошибка больше не повторяется



Диагностика 500 должна заканчиваться не только исправлением, но и regression test, который предотвращает повторение проблемы.

Мини-практика: диагностировать failing Product request

Найти причину ошибки GET /api/products/:id с помощью debugger и structured logs.

Задание

- Воспроизвести failure
- Прочитать stack trace
- Поставить breakpoint в Controller/Service
- Проверить id и результат repository
- Добавить requestId middleware
- Добавить logger.error в error handler
- Исправить причину
- Запустить integration test

Критерии проверки

- Причина объяснена конкретной строкой/условием
- Клиент не получает stack trace
- Server log содержит requestId
- Regression test подтверждает исправление

Checklist production-safe logging

- ✓ Используются уровни log
- ✓ Logs имеют стабильную структуру
- ✓ Есть requestId
- ✓ Ошибки логируются с Error object
- ✓ Client получает безопасное сообщение
- ✓ Passwords/tokens redacted
- ✓ User input не становится бесконтрольными top-level fields
- ✓ Development и production logging различаются только presentation/level, а не смыслом
- ✓ 500 можно связать с конкретным request
- ✓ После исправления добавляется regression test

Контрольные вопросы

1. Чем debugging отличается от logging?
2. Что показывает stack trace?
3. Чем `--inspect-brk` отличается от `--inspect`?
4. Что такое breakpoint?
5. Зачем structured logging?

6. Что означают уровни `info/warn/error`?
7. Зачем `requestId`?
8. Почему нельзя логировать `Authorization` header целиком?
9. Что должен получить клиент при `internal 500`?
10. Почему после исправления ошибки нужен regression test?

Итоги занятия

- 1 Начинаяте диагностику с воспроизведения и фактов.
- 2 Stack trace помогает локализовать ошибку.
- 3 Node Inspector позволяет ставить breakpoints и исследовать runtime state.
- 4 Structured JSON logs удобнее свободного console.log.
- 5 Log levels отделяют обычные события от ошибок.
- 6 Request ID связывает события одного HTTP request.
- 7 Child logger переносит context между слоями.
- 8 Secrets и tokens должны быть redacted.
- 9 Internal stack нужен серверу, но не клиенту.
- 10 Исправленная ошибка должна получить regression test.